



# Scenariusz obrony live demo

## Cel prezentacji

Dla kazdej podatnosci pokazujemy ten sam schemat:

1. Atak dziala w trybie vulnerable.
2. Ten sam atak nie dziala w trybie secure.
3. Pokazujemy konkretna roznice w kodzie.

Zakres dla zespolu 2-osobowego:

- Minimum wymagane: SQL Injection, XSS + 3 dodatkowe.
- Finalny zakres projektu: 8 podatnosci, czyli minimum plus bonusy.

## Finalna kolejnosc demo

1. SQL Injection
2. Stored XSS
3. Broken Authentication
4. Broken Access Control / IDOR
5. CSRF
6. Sensitive Data Exposure
7. Path Traversal / LFI
8. Command Injection

## Przygotowanie

Przed demo uruchom:

```
go mod tidy
npm install
npm run build:css
go test -tags=integration .
```

Tryb vulnerable:

```
SECURITY_ENABLED=false go run .
```

Tryb secure:

```
SECURITY_ENABLED=true go run .
```

Warto usunac `app.db` przy przełączaniu trybu, jeżeli potrzebna jest czysta baza seed:

```
rm app.db
```

W UI można też przełączać tryb bez restartu przyciskiem w nagłówku:

- w vulnerable mode: ↔ Secure
- w secure mode: ↔ Vulnerable

Przycisk wysyła `POST /ui/mode/toggle` i wraca na aktualną stronę. Do demo zależności od danych zapisanych w bazie, np. plaintext-vs-bcrypt, nadal najlepiej użyć czystej bazy po restarcie.

Zrzuty ekranów do sprawozdania są w `docs/screenshots/`.

## Podział ról

- Osoba 1: wykonuje ataki w przeglądarce/curl/Burp.
- Osoba 2: tłumaczy, co się dzieje i pokazuje kod.

## 1. SQL Injection

Vulnerable:

```
curl "http://localhost:8080/api/search?q=' OR 1=1 --"
curl "http://localhost:8080/api/search-vulnerable?q=zz' UNION SELECT id, username, password"
```

Secure:

```
SECURITY_ENABLED=true go run .  
curl "http://localhost:8080/api/search?q=' OR 1=1 --"
```

Co pokazac w kodzie:

- `internal/service/service.go`
- `SearchPostsVulnerable` - konkatencja SQL.
- `SearchPostsSecure` - parametry `?`.

Narracja:

Dane uzytkownika w vulnerable staja sie fragmentem SQL. W secure sa tylko wartoscia parametru.

## 2. Stored XSS

Vulnerable:

```
curl -X POST http://localhost:8080/api/comments-vulnerable \  
-H "Content-Type: application/json" \  
-d '{"post_id":1,"body":"<img src=x onerror=\\"alert(1)\\>","author":"attacker"}'
```

UI:

```
/ui/posts/view/1  
payload: <script>alert('XSS')</script>
```

Secure:

```
SECURITY_ENABLED=true  
/ui/posts/view/1  
ten sam payload w komentarzu
```

Co pokazac w kodzie:

- `internal/views/pages.templ` - `CommentsList` , raw render tylko w vulnerable.
- `internal/service/service.go` - `stripUnsafeHTML` .

Narracja:

W vulnerable przeglądarka dostaje wykonywalny HTML/JS. W secure dostaje tekst po sanityzacji i escapingu.

## 3. Broken Authentication

Vulnerable:

```
curl -i -X POST http://localhost:8080/login \  
-H "Content-Type: application/json" \  
-d '{"username":"admin","password":"wrong"}'
```

Secure:

```
SECURITY_ENABLED=true go run .  
curl -i -X POST http://localhost:8080/login \  
-H "Content-Type: application/json" \  
-d '{"username":"admin","password":"wrong"}'
```

Co pokazać w kodzie:

- `internal/handlers/handlers.go` - `evaluateLogin` .
- `internal/service/service.go` - `ValidateUserCredentials` .

Narracja:

W vulnerable sprawdzane jest tylko istnienie użytkownika. W secure hasło musi pasować do bcrypt hash.

## 4. Broken Access Control / IDOR

Vulnerable:

1. Uruchom `SECURITY_ENABLED=false` .

2. Zaloguj sie jako `user1` z dowolnym haslem.
3. Wejdz na `/ui/idor-demo`.
4. Usun post nalezacy do `admin`.

Secure:

1. Uruchom `SECURITY_ENABLED=true`.
2. Zaloguj sie jako `user1 / user1pass`.
3. Spróbuj usunac post admina.
4. Aplikacja blokuje operacje.

Co pokazac w kodzie:

- `internal/handlers/handlers.go` - `PagePostDelete`, `PostDelete`, `canDeletePost`.
- `internal/service/service.go` - `GetPostAuthor`, `IsUserAdmin`.

Narracja:

Samo ID rekordu nie wystarcza do autoryzacji. Secure mode sprawdza wlasciciela albo role admin.

## 5. CSRF

Vulnerable:

1. Zaloguj sie w aplikacji.
2. Wejdz na `/ui/csrf-demo`.
3. Zmien email bez tokena.
4. Pokaz przyklad zlosliwego formularza:

```
<form method="POST" action="http://localhost:8080/ui/csrf-demo">
  <input name="new_email" value="hacked@evil.com">
</form>
<script>document.forms[0].submit()</script>
```

Secure:

1. Wejdz na `/ui/csrf-secure`.
2. Pokaz hidden input `csrf_token`.

3. Wyslij POST bez tokena albo z blednym tokenem.
4. Serwer zwraca 403 .

Co pokazac w kodzie:

- `internal/handlers/handlers.go` - `CsrfFormVulnerable` , `CsrfSecureForm` .
- `internal/views/pages.templ` - `CSRFDemoPage` .

Narracja:

Cookie sesji nie jest dowodem intencji uzytkownika. Token CSRF dodaje drugi warunek dla akcji mutujacej.

## 6. Sensitive Data Exposure

Vulnerable:

```
sqlite3 app.db "SELECT username, password_hash, email FROM users;"
```

Albo:

```
/ui/db-expose
```

Secure:

```
SECURITY_ENABLED=true go run .  
sqlite3 app.db "SELECT username, password_hash, email FROM users;"
```

Co pokazac w kodzie:

- `internal/service/service.go` - `preparePassword` .
- `internal/db/db.go` - `encodePasswordForSeed` .
- `internal/views/pages.templ` - `DBExposePage` .

Narracja:

W vulnerable wyciek bazy oznacza wyciek hasel. W secure wyciek bazy ujawnia tylko hashe bcrypt.

## 7. Path Traversal / LFI

Vulnerable:

```
curl "http://localhost:8080/api/files-vulnerable?name=../go.mod"
curl "http://localhost:8080/api/files-vulnerable?name=../app.db" | xxd | head -4
```

UI:

```
/ui/path-traversal?name=../go.mod
```

Secure:

```
curl "http://localhost:8080/api/files-secure?name=../go.mod"
```

Oczekiwany wynik secure:

```
{"error":"path traversal detected – access denied"}
```

Co pokazać w kodzie:

- `internal/handlers/handlers.go` - `FilesVulnerable` , `FilesSecure` , `safeUploadPath` .
- `internal/views/pages.tmpl` - `PathTraversalPage` .

Narracja:

Vulnerable łączy `./uploads/` z inputem. Secure odrzuca `..` , ścieżki absolutne i wszystko, co wychodzi poza katalog uploadów.

## 8. Command Injection

Vulnerable:

```
curl "http://localhost:8080/api/ping-vulnerable?host=127.0.0.1%3B%20whoami"
curl "http://localhost:8080/api/ping-vulnerable?host=127.0.0.1%3B%20id"
```

UI:

```
/ui/cmd-injection?host=127.0.0.1+%3B+whoami
```

Secure:

```
curl "http://localhost:8080/api/ping-secure?host=127.0.0.1%3B%20whoami"
```

Oczekiwany wynik secure:

```
{"error":"invalid host: only [a-zA-Z0-9.-] allowed"}
```

Co pokazać w kodzie:

- `internal/handlers/handlers.go` - `PingVulnerable` , `PingSecure` , `validHostRE` .
- `internal/views/pages.templ` - `CmdInjectionPage` .

Narracja:

Vulnerable przekazuje input do `sh -c` , więc separator `;` uruchamia kolejną komendę.

Secure nie używa shella i waliduje host.

## Finalna checklista przed obroną

- ☐ Działa `go test -tags=integration` .
- ☐ Działa `go build -o /tmp/bai_check_app main.go`
- ☐ Mamy otwarte karty UI dla wszystkich demo.
- ☐ Mamy terminal z curl payloadami.
- ☐ Wiemy, które funkcje pokazać w code review.
- ☐ Po zmianie `SECURITY_ENABLED` wiemy, czy usuwamy `app.db` dla czystego seedu.